

SWAYAM NPTEL

ONLINE CERTIFICATION COURSE ON

Statistical and Machine Learning Methods: Basics and Applications

MODULE 10 : Decision Tree, Random Forest and Gradient Boosting

W10L1 : Decision Trees: Part I

BY

Prof. Rajib Maity

Professor, Department of Civil Engineering
Chairperson, School of Water Resources
Indian Institute of Technology Kharagpur
Kharagpur – 721302, West Bengal

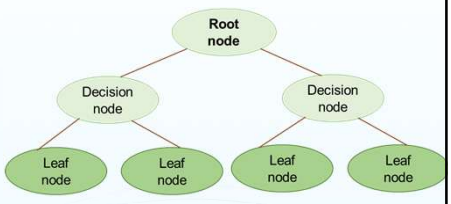
CONCEPTS COVERED

Decision Trees: Part I

NPTEL

Introduction to Decision Trees

- A decision tree is a decision-support method that represents a sequence of decisions and their possible consequences using a tree-like structure.
- Because of its flowchart form, the method allows different decision paths to be followed systematically towards a required objective.
- Decision trees have therefore been commonly applied in operations research for analysing decisions and identifying suitable paths towards an objective.



Introduction to Decision Trees

- In statistics and machine learning, decision trees are widely used for addressing non-linear classification and regression problems.
- An important advantage of the tree structure is its interpretability, since the sequence of decisions leading to an outcome can be followed directly.
- Decision trees are comparatively simple models, and are therefore, commonly considered as weak learners in machine learning.
- More complex and generally more accurate models, such as neural networks and support vector machines, are regarded as strong learners.

Decision Trees

- An individual decision tree is a relatively simple model and may have lower predictive capability than more complex learning models.
- However, many weak learners can work collectively and achieve strong predictive performance when used as an ensemble.
- Thus, although an individual decision tree is relatively simple, a large collection of such weak learners can form a highly effective predictive model.

Classification and Regression Trees (CART)

- Decision tree methods partition the predictor x-space into rectangular regions and fit a simple function $y = f(x)$ to the response variable within each region.
- The most commonly used decision tree approach is the Classification and Regression Tree (CART) method.
- In CART, $f(x)$ is taken as a constant within each region, resulting in a step change at the boundary between two regions.
- Although this representation is simple, CART is useful because it provides an intuitive view of how the response variable broadly depends on the predictors.
- When a large number of predictors are available, CART also offers a computationally inexpensive way to identify a smaller set of relevant predictors.

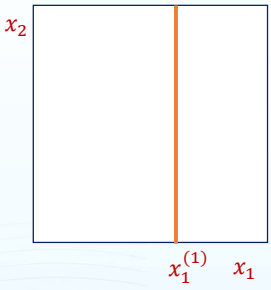
Classification and Regression Trees (CART)

- These selected predictors can subsequently be used in more accurate but computationally demanding models, such as MLP neural networks and SVMs.
- However, CART can be quite unstable when noise is present in the data.
- As CART can be applied to both non-linear classification and regression problems. Let's consider first a regression problem with target variable y.
- For simplicity, let the predictor vector x contains only two variables, x_1 and x_2.
- CART searches for a split point $x_1^{(1)}$ (where the superscript indicates the order of the split) that divides the predictor space into two separate regions.
- The fitted response is represented by a step function, with a constant prediction assigned within each region:

$$f(x) = c_1, \qquad x_1 < x_1^{(1)}$$
$$f(x) = c_2, \qquad x_1 \geq x_1^{(1)}$$

Classification and Regression Trees (CART)

- When the fit is evaluated using mean squared error (MSE), c_1 and c_2 are the mean values of y within the two regions.
- A similar search is performed along predictor x_2 to obtain a candidate split point $x_2^{(1)}$.
- The first split is selected from $x_1^{(1)}$ and $x_2^{(1)}$ based on which candidate produces the smaller MSE.



Example:

$PM_{2.5}$ concentration, y $(\mu g/m^3)$ is predicted using:

x_1: PM_{10} concentration, $T(\mu g/m^3)$

x_2: relative humidity, RH $(\%)$

The observations are tabulated below.

Observation	x_1: PM_{10} $(\mu g/m^3)$	x_2: RH $(\%)$	y: $PM_{2.5}$ $(\mu g/m^3)$
1	20	40	5
2	22	70	9
3	24	45	6
4	26	75	10
5	40	35	20
6	42	65	21
7	55	50	30
8	58	80	31

Solution

Search along x_1:

- The possible split locations along x_1 occur midway between consecutive PM_{10} values.
- For this dataset, the best split along x_1 occurs at $x_1^{(1)} = 33$ because 33 lies midway between 26 and 40.

Region 1:

$x_1 < 33$ contains $y = \{5,9,6,10\}$.

Therefore, $c_1 = \frac{5+9+6+10}{4} = 7.5$

MSE in this region:

$$E_1 = \frac{(5-7.5)^2 + (9-7.5)^2 + (6-7.5)^2 + (10-7.5)^2}{4} = 4.25$$

NPTEL

Solution:

Region 2:

$x_1 \geq 33$ contains $y = \{20,21,30,31\}$.

Therefore, $c_2 = \frac{20+21+30+31}{4} = 25.5$

MSE in this region:

$$E_1 = \frac{(20-25.5)^2 + (21-25.5)^2 + (30-25.5)^2 + (31-25.5)^2}{4} = 25.25$$

Hence, the overall MSE after this split:

$$MSE_{x_1} = \frac{4(4.25) + 4(25.25)}{8}$$

$$MSE_{x_1} = 14.75$$

Solution:

Comparison with another split along x_2:

A similar search is performed along relative humidity x_2.

For this dataset, the best split along x_2 occurs at: $x_2^{(1)} = 77.5$

The resulting overall MSE is:

$$MSE_{x_2} = 65.71$$

Since, $14.75 < 65.71$, the first CART split is selected along x_1 i.e., $x_1^{(1)} = 33$.

Thus, first split is made along x_1 because it produces the smaller MSE.

Solution:

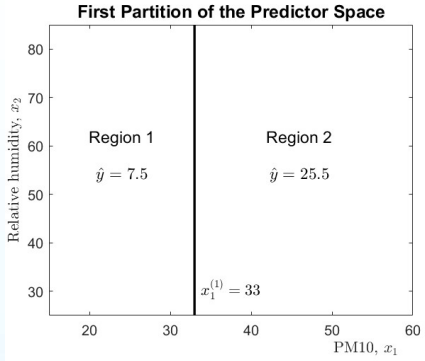
After the first split, $x_1 = 33$, the predictor space is divided into two rectangular regions.

Region 1:

$$x_1 < 33, \qquad \hat{y} = c_1 = 7.5$$

Region 2:

$$x_1 \geq 33, \qquad \hat{y} = c_2 = 25.5$$



- The vertical split at $x_1 = x_1^{(1)}$ creates two regions in the predictor space.
- Within each region, CART assigns a single constant value of $\hat{y}$.
- Therefore, the fitted response changes from one constant value to another when the split boundary is crossed.

Solution:

Selecting the second split:

- The splitting procedure is now repeated. CART again examines possible splits along both x_1 and x_2 within the existing regions.
- The largest improvement is obtained by splitting the right-hand region: $x_1 \geq 33$, again along x_1, at $x_1^{(2)} = 48.5$.

The right-hand region contains: $y = \{20,21,30,31\}$, with $\hat{y}$ =25.5 and $MSE = 25.25$.

For $33 \leq x_1 < 48.5$, the response values are: $y = \{20, 21\}$.

Therefore, $\hat{y} = \frac{20+21}{2} = 20.5$ and $MSE = 0.25$

For $x_1 \geq 48.5$, the response values are 30, 31.

Therefore, $\hat{y} = \frac{30+3}{2} = 30.5$ and $MSE = 0.25$

The predictor space now contains three regions.

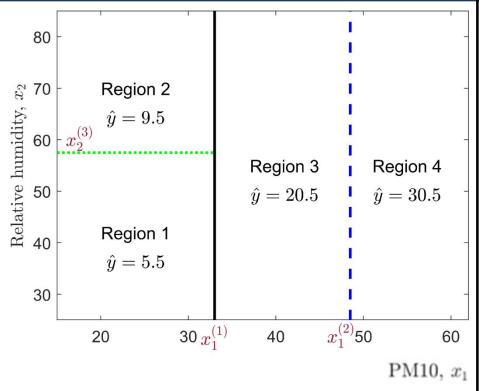
Solution:

Selecting the third split:

The splitting procedure is repeated once again.

The remaining left-hand region: $x_1 < 33$, contains $y = \{5,9,6,10\}$.

CART now selects a split along x_2: $x_2^{(3)} = 57.5\%$



- **Lower x_2 region:** For $x_2 < 57.5$, the response values are 5,6.

  Thus , $\hat{y} = 5.5$.
- **Higher x_2 region:** For $x_2 \geq 57.5$, the response values are 9,10.

  Thus , $\hat{y} = 9.5$.

  Both regions have $MSE = 0.25$.

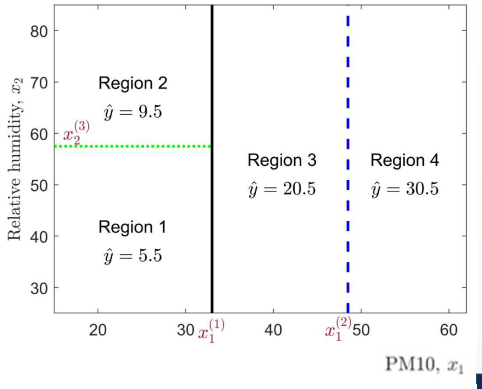
Solution:

After three splits, i.e.,

$x_1^{(1)} = 33$;

$x_1^{(2)} = 48.5$;

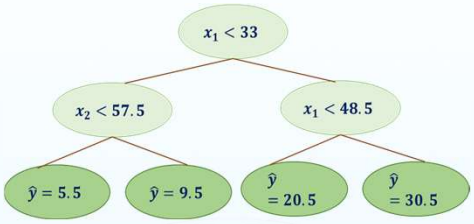
$x_2^{(3)} = 57.5$



Region	$\hat{y}$
Region 1: $x_1 < 33, x_2 < 57.5$	5.5
Region 2: $x_1 < 33, x_2 \geq 57.5$	9.5
Region 3: $33 \leq x_1 < 48.5$	20.5
Region 4: $x_1 \geq 48.5$	30.5

Representation as a Regression Tree

- The same four predictor-space regions can be represented by a regression tree.
- Each internal node contains a condition used to split the observations into two groups.
- If the condition is satisfied, the left branch is followed; otherwise, the right branch is followed.
- Each terminal or leaf node represents one of the regions formed in the predictor space.
- The model output $\hat{y}$ at a leaf node is the mean target value of y observations in that region.
- The number of observations n belonging to each region can also be displayed at the corresponding leaf node.



Further growth of Regression Tree

- The splitting process need not stop after the first three splits.
- Additional splits further divide the existing regions and increase the number of terminal nodes in the tree.
- Each additional terminal node represents a new partitioned region with its own constant value of $\hat{y}$.
- Splitting can continue until a specified stopping criterion is reached.
- For the same example, Regions 3 and 4 still contain two observations each.

Existing Region 3: $\mathbf{33 \le x_1 < 48.5}$

Continue splitting at $x_2^{(4)} = 50$

This gives:

$$\boldsymbol{x_2 < 50} \Rightarrow \hat{y} = 20, n = 1$$

$$\boldsymbol{x_2 \ge 50} \Rightarrow \hat{y} = 21, n = 1$$

Further growth of Regression Tree

Existing Region 4: $x_1 \geq 48.5$

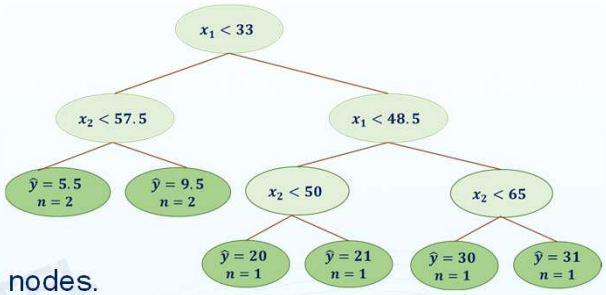
A further split can be made at $x_2^{(5)} = 65$

This gives:

$$x_2 < 65 \Rightarrow \hat{y} = 30, n = 1$$

$$x_2 \geq 65 \Rightarrow \hat{y} = 31, n = 1$$

The regression tree now contains six terminal nodes.



- Five internal splits divide the predictor space into six regions represented by six terminal nodes.
- Every leaf contains a constant model output $\hat{y}$, calculated from the mean target value within that region.
- As the tree grows, the regions become smaller and the fitted values can follow the training observations more closely.

Pruning using Regularization

- A tree should not necessarily stop growing when one split produces only a small reduction in MSE.
- Although an individual split may provide little improvement, a subsequent split can produce a much larger decrease in MSE.
- Therefore, CART first allows the tree to grow to a relatively large size.
- The resulting large tree is subsequently reduced to an appropriate size through pruning using a regularization penalty.

Let L = no. of leaf nodes

For a regression tree containing L leaves,

$$J(L) = E(L) + \lambda L$$

where, $E(L)$ is the overall MSE of the tree containing L leaf nodes; λ is the weight-penalty parameter; λL penalizes trees containing an excessive number of leaf nodes.

Pruning using Regularization

- The objective function therefore accounts for both prediction error and tree-size penalty.
- A larger tree may reduce $E(L)$, but it also increases the penalty term λL.
- The preferred tree for a specified λ is the one having the smallest $J(L)$.

The pruning procedure begins with the fully grown regression tree.

- Start with the full tree containing the largest number of leaf nodes.
- Remove the internal node whose removal produces the smallest increase in MSE.
- Continue removing internal nodes in this manner to obtain progressively smaller trees.
- This produces a sequence of candidate trees containing different numbers of leaf nodes L.

Pruning using Regularization

- Calculate $J(L) = E(L) + \lambda L$ for every candidate tree.
- For the specified λ, select the tree having the smallest value of $J(L)$.

For the same example,

For the six-leaf tree:

$$E(6) = 0.125$$

If one of the additional splits is removed:

$$E(5) = 0.1875$$

Removing the other additional split returns us to the four-leaf tree:

$$E(4) = 0.25$$

Further pruning gives:

$$E(3) = 2.25$$

And with only two leaves, $E(2) = 14.75$

Pruning the Regression Tree

- Let $\lambda = 0.5$, then

L	$E(L)$	λL	$J(L) = E(L) + \lambda L$
6	0.125	3.00	3.125
5	0.187	2.50	2.687
4	0.250	2.00	2.250
3	2.250	1.50	3.750
2	14.750	1.00	15.750

Therefore, $L = 4$ gives the minimum objective value for $\lambda = 0.5$.

Selecting the Penalty Parameter λ

- Different values of λ produce different balances between prediction error and tree size.
- The appropriate value of λ is determined using cross-validation.
- CART is evaluated using several values of λ, and a pruned tree is obtained for each value.
- The corresponding cross-validated MSE values are then compared.
- The value of λ producing the smallest cross-validated MSE is selected.

NPTEL


FREE ONLINE EDUCATION
swayam
Thank You!
alt



SWAYAM NPTEL

ONLINE CERTIFICATION COURSE ON

Statistical and Machine Learning Methods: Basics and Applications

MODULE 10 : Decision Tree, Random Forest and Gradient Boosting

W10L2 : Decision Trees: Part - II

BY

Prof. Rajib Maity

Professor, Department of Civil Engineering
Chairperson, School of Water Resources
Indian Institute of Technology Kharagpur
Kharagpur – 721302, West Bengal

CONCEPTS COVERED

Decision Trees: Part - II

NPTEL

CART for Classification

- CART can also be applied for classification problems, i.e., when the target variable y represents one of K discrete classes.
- After partitioning the predictor $\mathbf{x}$-space, each region is assigned to one class C_k, where $k = 1, \ldots, K$.
- The predicted class $\hat{y}$ for a region is the class containing the largest number of target observations in that region.
- Unlike regression CART, tree growth is not based on minimizing MSE because the response variable is categorical.
- Instead, the quantity E represents the classification impurity, which is minimized while constructing the tree.

CART for Classification

- The primary goal of a classification tree is to achieve maximum purity across leaf nodes, with each node representing a unique class.
- Consequently, the algorithm focuses on minimizing impurity throughout the tree construction process.
- CART therefore searches for splits that produce regions having lower classification impurity.
- Two commonly used measures are:
  - Entropy impurity
  - Gini impurity

Entropy Impurity

- Suppose the tree contains L regions and there are K response classes.
- Let p_{lk} denote the fraction of observations in region l that belong to class k.
- The entropy impurity (also known as cross-entropy or deviance) is:

$$E(L) = \sum_{l=1}^{L} \sum_{k=1}^{K} -p_{lk} \, log(p_{lk})$$

where the logarithm is often taken as $\log_2$

- If almost all observations in a region belong to one class, its entropy impurity is small.
- Impurity increases as observations become more evenly distributed among the different classes within the region.

Gini Impurity Index

- Another commonly used measure of classification impurity is the Gini impurity index:

$$E(L) = \sum_{l=1}^{L} \sum_{k=1}^{K} p_{lk} \, (1 - p_{lk})$$

where p_{lk} represents the fraction of observations in region l belonging to class k.

- Gini impurity becomes small when one class dominates within a region.
- Hence, CART can select splits by searching for reductions in the classification impurity.

Example

Consider an illustrative dataset in which monthly drought severity has been classified as Mild or Severe. Monthly precipitation (P) and Potential Evapotranspiration (PET) are used as predictors to demonstrate CART classification.

Obs.	P (mm/month)	PET (mm/month)	Drought Class
1	72	128	Mild
2	65	142	Mild
3	58	135	Mild
4	48	150	Mild
5	52	145	Severe
6	39	153	Severe
7	32	140	Severe
8	25	158	Severe

Determine the first CART split using the Gini impurity and compare the best splits obtained from P and PET.

Solution:

Candidate splits using Precipitation

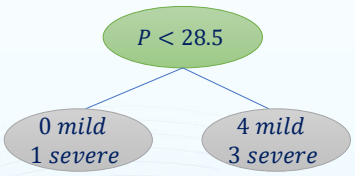
Arrange observations in ascending order of precipitation:

25, 32, 39, 48, 52, 58, 65 and 72

Candidate split points are taken midway between successive precipitation values:

28.5, 35.5, 43.5, 50, 55, 61.5 and 68.5

The weighted Gini impurity for the first split, $P < 28.5$



The child nodes are:

Child node	Mild	Severe	p_{Mild}	p_{Severe}
Left	0	1	$\frac{0}{1} = 0$	$\frac{1}{1} = 1$
Right	4	3	$\frac{4}{7} = 0.571$	$\frac{3}{7} = 0.429$

Solution:

Gini Impurity:

$$E(L) = \sum_{l=1}^{l} \sum_{k=1}^{K} p_{lk}\,(1 - p_{lk})$$

Gini impurity in the left node,

$G_L = p_{Mild}(1 - p_{Mild}) + p_{Severe}(1 - p_{Severe}) = 0(1-0) + 1(1-1) = 0$

Similarly, $G_R = \frac{3}{7}\left(1 - \frac{3}{7}\right) + \frac{4}{7}\left(1 - \frac{4}{7}\right) = 0.49$

The weighted Gini impurity $= \frac{1}{8}G_L + \frac{7}{8}G_R = \frac{1}{8}(0) + \frac{7}{8}(0.49) = 0.429$

The same procedure is repeated for all remaining split points (tabulated in the next slide).

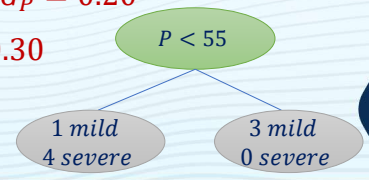
Solution:

- Therefore, minimum weighted Gini = 0.20 obtained at $P < 43.5$ and $P < 55$.
- $P < 55$ is used for subsequent illustration.
- At the root node, before any split, all 8 observations are together (on one node): 4 Mild, 4 Severe
- So, $p_{Mild} = \frac{4}{8} = 0.5$; $p_{Severe} = \frac{4}{8} = 0.5$

  $G_{root} = 0.5(1 - 0.5) + 0.5(1 - 0.5) = 0.50$
- Weighted Gini after a split at $P < 55$, $G_P = 0.20$

  $\Delta G = G_{root} - G_P = 0.50 - 0.20 = 0.30$

Split	Weighted Gini
(P<28.5)	0.429
(P<35.5)	0.333
(P<43.5)	**0.200**
(P<50)	0.375
(P<55.0)	**0.200**
(P<61.5)	0.333
(P<68.5)	0.429



Solution:

Candidate Splits Using PET

Arrange observations in ascending order of PET:

128,135,140,142,145,150,153 and 158

Candidate split points are taken midway between successive precipitation values:

131.5,137.5,141,143.5,147.5,151.5,155.5

The weighted Gini impurity is calculated across all splits similar to Precipitation.

The minimum weighted Gini impurity occurs at PET<137.5.

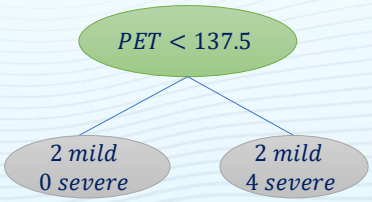
The resulting child nodes at PET < 137.5 are:

Left child:

PET<137.5: 2 Mild, 0 Severe

Right child:

PET≥ 137.5: 2 Mild, 4 Severe



Solution:

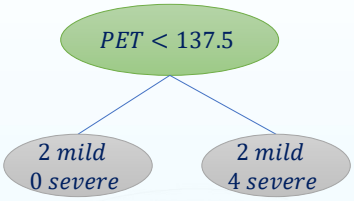
Candidate Splits Using PET

For left child,

$$G_L = 1(1-1) + 0(1-0) = 0$$

Similarly, for the right child,

$$G_R = \frac{2}{6}(1-\frac{2}{6}) + \frac{4}{6}\left(1-\frac{4}{6}\right) = 0.44$$



Weighted Gini impurity:

$$G_{PET} = \frac{2}{8}(0) + \frac{6}{8}(0.44) = 0.33$$

$$\Delta G = G_{root} - G_{PET} = 0.50 - 033 = 0.17$$

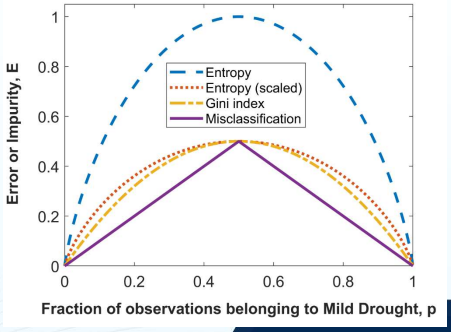
Since G_P (0.20) < G_{PET} (0.33), CART selects precipitation for the first split.

Impurity Measures in Binary Classification

- During the pruning stage of the tree, E in the regularized objective is generally represented by the misclassification rate, i.e., the fraction of incorrectly classified observations.
- Consider now a binary classification, $K = 2$, and a single leaf node.
- Let p denote the fraction of observations in that leaf node belonging to class 1; consequently, the fraction belonging to Class 2 is $1 - p$.
- The entropy impurity, Gini impurity, and misclassification rate can then be expressed as functions of p.
- When p approaches 0 or 1, one class dominates the leaf node and the corresponding error E becomes small.
- The largest impurity occurs when the observations are distributed nearly equally between the two classes in a region.

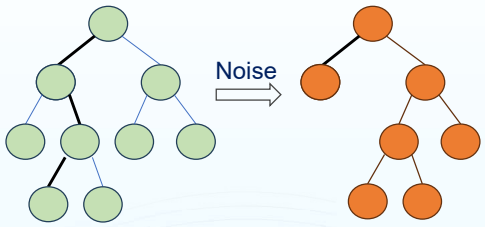
Impurity Measures in Binary Classification

- Since entropy and Gini impurity are differentiable functions, they are generally preferred against the misclassification rate during numerical optimization.
- The figure shows how entropy impurity, Gini impurity, and misclassification rate vary with the class fraction p in a binary leaf node. The example is shown with respect to the fraction of observations belonging to mild drought
- Entropy impurity, Gini impurity, and misclassification rate are shown as functions of p.
- A scaled entropy curve is also shown with the same maximum value as the Gini impurity, allowing easier comparison of their shapes.



Disadvantages of CART

- A decision tree is highly sensitive to small noise perturbations present in the training dataset.
- Even a minor change in the data may produce a different split during tree construction.
- When an initial split changes, all subsequent branches developed after that node may also change substantially.
- Different tree structures can produce different predictions for the same input, thereby reducing overall model stability.

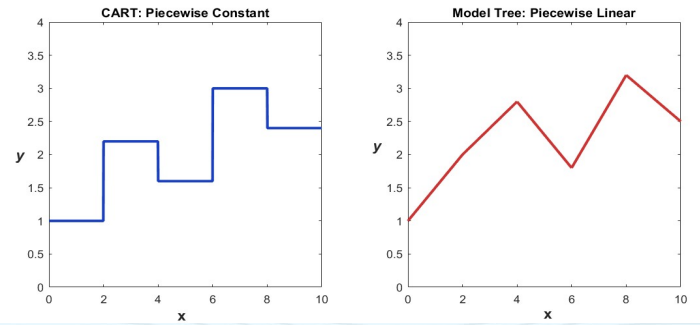


Limitations: CART Regression Output

- Within each partitioned region of the predictor space, CART assigns a constant prediction $\hat{y}$.
- Therefore, the predicted response changes in discrete steps when an observation moves from one region to another.
- This stepwise behaviour is not an issue for classification problem, because the response itself consists of discrete classes.
- However, for regression problems, the CART output is not ideal, as the continuous response is represented using piecewise constant values, which can create abrupt changes between neighbouring regions.
- CART may therefore be regarded as a zero-order model, since the non-linear relationship $\hat{y} = f(x)$ is represented using piecewise constant functions.

Limitations: CART Regression Output

- To move beyond this piecewise-constant representation, the non-linear relationship using piecewise linear functions are used between the partitioned regions.
- In the final stage, a smoothing procedure may also be applied to reduce sharp discontinuities between neighbouring linear models at the region boundaries.



Limitations: Axis-Parallel Splits

- Standard CART generally uses splitting rules of the form: $x_j < s$.
- Such splits produce decision boundaries that are parallel to one of the predictor axes.
- If the actual decision boundary is oblique, to any of the predictor axis, such as splitting in the form of $\sum_j a_j x_j < s$.
- These oblique splits are not restricted to directions parallel to the predictor axes.
- However, use of such general boundaries reduces the simplicity and intuitive interpretation.

Predictor importance in CART

- A CART model can also be used to assess how strongly different predictor variables contribute to the tree.
- For a regression tree:
  - Let us consider an internal node t containing N_t observations. The MSE at this node is

$$E_t = \frac{1}{N_t} \sum_{i \in S_t} (y_i - \bar{y}_t)^2$$

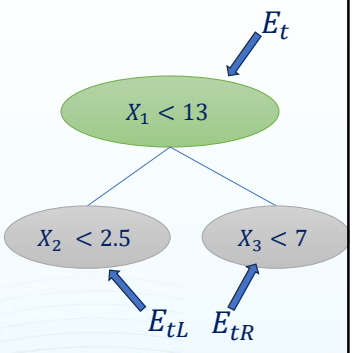
  - N_t is the number of observations contained in node t.
  - S_t represents the set of observations belonging to node t.
  - $\bar{y}_t$ is the mean of the response values y_i within node t.
  - E_t measures the prediction error associated with the observations in that node.
- Thus, before a node is divided, its response variation is represented through E_t.

NPTEL

Reduction in Error due to a Split

- Suppose internal node t is divided into a left descendant node and a right descendant node.
- The corresponding MSE values are E_{tL} for the left descendant node, and E_{tR} for the right descendant node.
- The reduction in MSE produced by splitting node t is

$$\Delta E_t = E_t - E_{tL} - E_{tR}$$



- A larger value of ΔE_t indicates that the split at node t produces a greater reduction in prediction error.
- Therefore, the error reduction associated with a split can be used to quantify the contribution of the predictor selected at that node.

NPTEL

Importance of a Predictor

- Let b_t denote the predictor selected for the best split at internal node t.
- For a tree containing L leaf nodes, there are $L-1$ internal nodes.
- The importance of predictor x_j is defined as

$$I_j = \sum_{t=1}^{L-1} \Delta E_t \Im(b_t = x_j)$$

where $\Im(b_t = x_j)$ is an indicator function that takes a value of 1 if the argument (■) is true and 0 if it is false.

- Therefore, only those internal nodes where predictor x_j is selected, contribute to I_j.
- The importance of a predictor is consequently obtained by accumulating the reductions in error from all splits made using that predictor.

Relative Importance of Predictors

- The importance of each variable is expressed relative to the predictor having the largest importance.
- The relative importance of $x_j = \frac{I_j}{\max(I_j)} \times 100$
- That is, the most important predictor is assigned a relative importance of 100% and the remaining predictors are expressed as percentages relative to this strongest predictor.
- For a classification tree:
  - E_t is evaluated using either Gini impurity or Entropy impurity (not through MSE at the internal node t.

Masking of Important Predictors

- An important predictor may not appear in the tree when another predictor contains very similar information but gives slightly better predictions.
- For example, suppose x_1 and x_3 contain similar information, with x_1 performing slightly better during tree construction.
- Then, CART may repeatedly select x_1 for the splits, while x_3 may not appear in the final tree.
- As a result, the calculated importance of x_3 may be very small or even zero, although x_3 still contains useful predictive information.
- This behaviour is referred to as masking of a predictor.
- If x_1 is removed from the predictor set, x_3 may then become prominent in the reconstructed tree, revealing its predictive value.
- Surrogate splits help to identify the importance of such masked predictors without removing the stronger predictor from the model.

Surrogate Splits in CART

- Also, there may be missing values of a predictor while a trained CART model is used for application with new observations.
- Two possible approaches are:
  - replace the missing value using the mean or median of that predictor or
  - use a surrogate split to continue through the tree.
- At an internal node t, surrogate splitting begins by defining a measure of similarity between the primary split s and an alternative split s'.
- Suppose the best split at node t is s, and this split uses predictor x_j.
- Next, alternative splits are considered using predictors other than x_j.

Surrogate Splits in CART

- Among these alternatives, the split s' having the greatest similarity with the primary split s is identified.
- This split s' is referred to as the best surrogate split for s.
- The same procedure can then be repeated to determine the second-best surrogate, followed by additional surrogate splits if required.
- Consider a prediction path that reaches a node whose primary split uses predictor x_j.
- If the value of x_j is missing for a particular observation, the primary split cannot be evaluated.
- A surrogate split based on another predictor x_k can then be used, provided it is the best available surrogate for the primary split.

Surrogate Splits in CART

- If both x_j and x_k are unavailable, the observation can proceed using the second-best surrogate split.
- In this way, surrogate splits allow prediction to continue even when the predictor required by the primary split is missing.

Similarity Measure for Surrogate Splits:

- Consider two splits at node t, the primary split s, based on predictor x_j, and a candidate surrogate split s', based on another predictor x_k.
- Let the two splitting rules be
  - $s: x_j < u$ and $s': x_k < v$.

Surrogate Splits in CART

- Let P_L denotes the fraction of observations at node t that satisfy $x_j < u$, and P_R denotes the fraction satisfying $x_j \geq u$.
- Here, L and R refer to the left and right branches from node t.
- Let P_{LjLk} represents the fraction of observations that goes to the left branch under both s and s'.
- Similarly, let P_{RjRk} represents the fraction of observations that goes to the right branch under both splits.
- When the surrogate split closely agrees with the primary split, $P_{LjLk} + P_{RjRk}$ approaches 1.

Note: If predictor x_k is negatively oriented relative to x_j, the surrogate split direction should be reversed (i.e., $-x_k < v$) so that the left and right branches remain consistent between s and s'.

Surrogate Splits in CART

- The similarity measure between the two splits is obtained as:

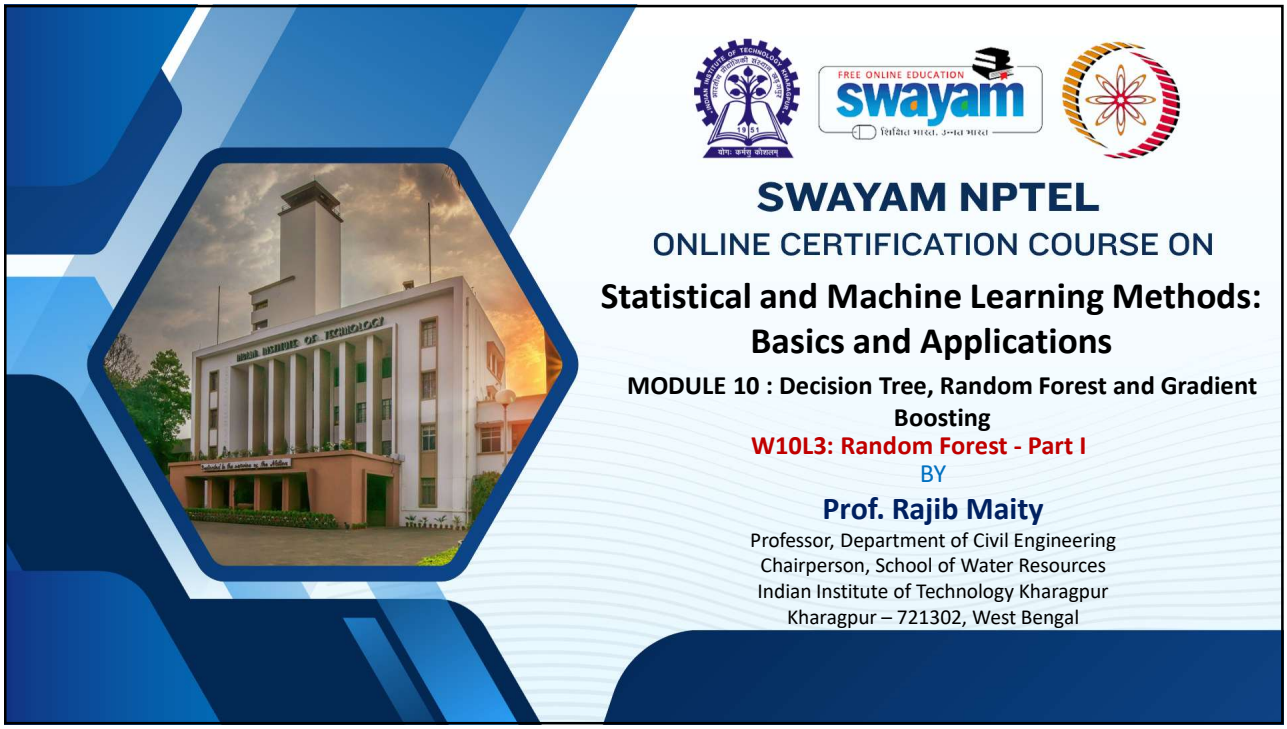
$$\lambda(s, s') = \frac{min(P_L, PR) - (1 - PLj_{Lk} - PR_{jRk})}{min(P_L, P_R)}$$

- Any candidate split s' for which $\lambda(s, s') < 0$ is discarded.
- Among the remaining candidates, the split with the largest value of $\lambda(s, s')$ is selected as the best surrogate split.

Surrogate Splits and Predictor Importance:

- Surrogate splits can also be used when evaluating the importance of predictor variables.
- Including surrogate splits in the predictor-importance calculation can help to identify the contribution of variables that are masked by other similar, slightly stronger predictors.

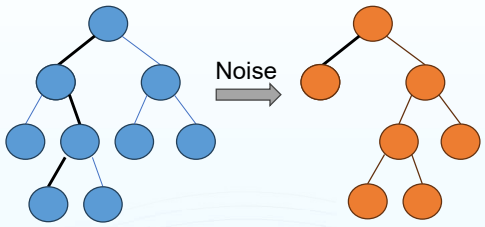

FREE ONLINE EDUCATION
swayam
Thank You!
alt


SWAYAM NPTEL
ONLINE CERTIFICATION COURSE ON
Statistical and Machine Learning Methods: Basics and Applications
MODULE 10 : Decision Tree, Random Forest and Gradient Boosting
W10L3: Random Forest - Part I
BY
Prof. Rajib Maity
Professor, Department of Civil Engineering
Chairperson, School of Water Resources
Indian Institute of Technology Kharagpur
Kharagpur – 721302, West Bengal


CONCEPTS COVERED
Random Forest: Part I
NPTEL

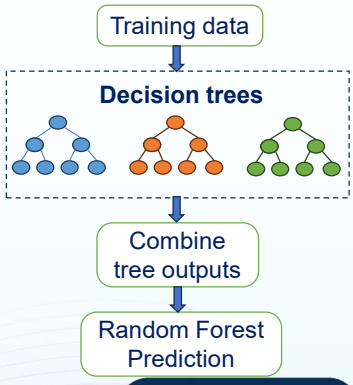
Background

- A decision tree is highly sensitive to small noise perturbations present in the training dataset.
- Even a minor change in the data may produce a different split during tree construction.
- When an upper-level split changes, all subsequent branches developed below that node may also change substantially.
- Different tree structures can produce different predictions for the same input, thereby reducing overall model stability.
- This limitation creates the need for an approach that does not depend entirely on one decision tree.



From a Single Tree to a Forest

- To improve the accuracy and stability of a decision-tree model, an ensemble of randomly constructed CART models is developed.
- This ensemble is called Random Forest model.
- Every tree develops its own structure and produces an individual prediction for the same input given to the Random Forest model.
- The individual tree outputs are combined to obtain one final prediction from the ensemble model.
- Classification predictions are combined through majority voting, whereas regression predictions are combined through numerical averaging.



Bagging – Bootstrapping

- An effective way to generate several models from a single model is through bagging which is abbreviated from Bootstrap AGGregatING.
- In bootstrap resampling, observations are randomly selected from the original dataset to create a new training dataset containing the same number of observations as the original dataset.
- Because sampling is performed with replacement, the same observation may be selected more than once within a bootstrap training dataset.
- On average, approximately 63.2% of the original observations are included, while the remaining 36.8% are not selected.
- This resampling procedure is repeated many times to generate a large collection of bootstrap training datasets.

Bagging – Bootstrapping

- During each random selection, the predictor values and their corresponding response value are drawn together as a pair.
- For serially correlated data, consecutive data segments covering at least the correlation time interval are sampled instead of individual observations.
- In bagging, one model is built from each training dataset generated through bootstrap resampling.
- Thus, from the multiple training datasets, an ensemble of models is obtained

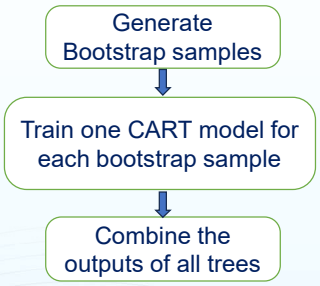
Random Forest

- Random Forest creates an ensemble by applying bagging, or bootstrap aggregation, to several CART models.
- However, training identical CART models on different bootstrap samples may not reduce model variance sufficiently.
- This limitation arises because the predictions produced by individual ensemble members can remain highly correlated.
- Random Forest therefore selects a random subset of the available predictors during the construction of tree.
- This additional randomization increases diversity among trees and improves the effectiveness of the ensemble.

Random Forest: Algorithm

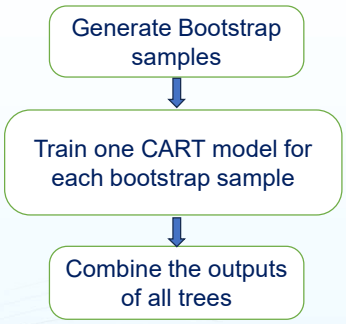
Let N denote the number of training data points, and let m represent the total number of predictor variables, the algorithm proceeds as follows:

- Create B bootstrap samples by drawing N training data points with replacement and repeating this sampling process B times.
- For every bootstrap sample, construct a CART model by repeating the following steps (in the next slide) at each terminal node until the minimum leaf-node size n_{min}, or another stopping criterion, is reached:



Random Forest: Algorithm

- Randomly choose p predictors from the original set of m predictors, where $1 \leq p < m$.
- Among these p selected predictors, identify the best splitting dimension and divide the current node into two new leaf nodes.



Output of Random Forest

- The Random Forest model output is obtained by averaging the CART outputs from all the B ensemble members in regression problems.
- In classification problems, the Random Forest model output is selected as class k chosen by the largest number of ensemble members in classification problems.

Hyperparameters

- The primary tuning hyperparameters in Random Forest are:
  - the number of predictors selected at each split, p.
  - the minimum number of observations permitted within a leaf node, n_{min}.
- For regression problems, the commonly used default sets p as the largest integer not exceeding $m/3$, with $n_{min} = 5$.
- For classification problems, the usual default sets p as the largest integer not exceeding $\sqrt{m}$, with $n_{min} = 1$.
- In practical applications, both hyperparameters are generally tuned through validation to obtain improved model performance.

Out-of-Bag (OOB) data

- During bootstrap sampling, approximately 36.8% of the original observations are not selected from a given bootstrap sample.
- These unselected data are called out-of-bag (OOB) data because they are not used to train the corresponding tree.
- Since the OOB observations are not used to train that tree, they can be used as validation data.
- The prediction error calculated from OOB data helps identify suitable values for p and n_{min}.
- Thus, OOB data eliminates the need for a separate validation dataset.

Number of Trees and OOB Error Stabilization

- The hyperparameter B, representing the number of trees in the ensemble, generally requires minimal tuning.
- As more trees are added, the OOB error decreases initially and eventually reaches an almost constant value beyond a threshold B_0.
- After this point, using $B > B_0$ provides negligible improvement in OOB error, indicating that the model does not overfit by adding extra trees.
- In practice, 100–200 trees are commonly sufficient, although a larger number can be used when computational resources are available.
- The forest can be grown sequentially by adding one tree at a time, and the process may be stopped once the OOB error has stabilized.

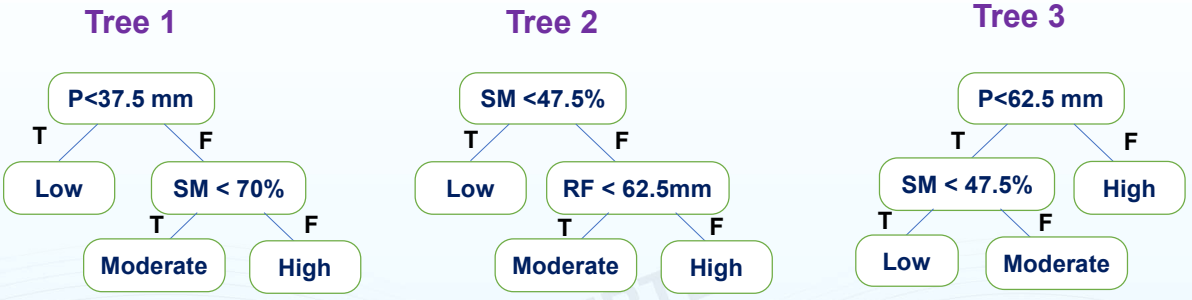
Example

A small Random Forest is developed to classify catchment flood potential using daily observations of three-day rainfall (P) and antecedent soil moisture (SM). The available training dataset contains six observations belonging to Low, Moderate, and High flood-potential classes.

Observation	P, (x_1) (mm)	SM, (x_2) (%)	Flood-potential class
1	20	30	Low
2	30	40	Low
3	45	55	Moderate
4	55	65	Moderate
5	70	75	High
6	85	85	High

The illustrative Random Forest contains three trees, thus $N = 6$, $m = 2$ and $B = 3$. For classification, $p = \sqrt{m} = \sqrt{2} \cong 1$

Example



where, T = True and F = False.

The following bootstrap sample are used:

Tree	Bootstrap sample IDs
Tree 1	1, 2, 2, 3, 5, 6
Tree 2	1, 3, 4, 4, 5, 6
Tree 3	2, 3, 3, 4, 5, 6

Example

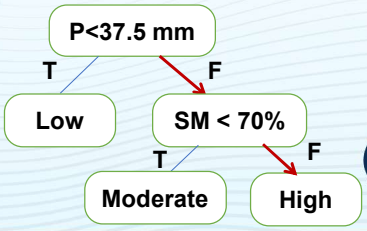
On a day, the following conditions are observed for the same catchment:
Three-day rainfall, $P = 60\ mm$
Antecedent soil moisture, $SM = 72\%$
Using the three trained decision trees, determine the flood-potential class predicted for this day and obtain the final Random Forest prediction through majority voting.

Solution

For, $P = 60\ mm$ and $SM = 72\%$

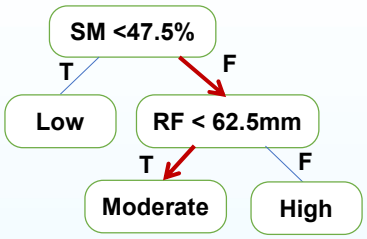
Tree 1:

$60 < 37.5$ is false.
Therefore, move to the soil-moisture condition: $72 < 70$ is false
Hence, Tree 1 prediction is High



Solution

For, $P = 60\ mm$ and $SM = 72\%$



Tree 2:

$72 < 47.5$ is false.

Therefore, move to the rainfall condition: $60 < 62.5$ is true

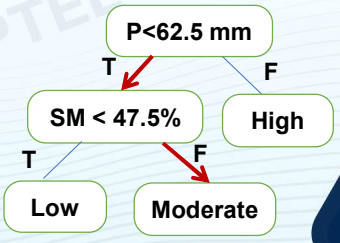
Hence, Tree 2 prediction is Moderate.

Tree 3:

$60 < 62.5$ is true.

Therefore, move to the rainfall condition: $72 < 47.5$ is false

Hence, Tree 3 prediction is Moderate.



Solution

Random Forest Prediction:

Tree	Predicted Class
Tree 1	High
Tree 2	Moderate
Tree 3	Moderate

Class	No. of votes
Low	0
Moderate	2
High	1

- For classification problems, the Random Forest output is the class k receiving the largest number of votes from the ensemble members; therefore, individual tree predictions are combined through majority voting.
- The Moderate class receives the largest number of votes among the three ensemble members.
- Therefore, the final Random forest prediction is Moderate flood potential.

Solution

OOB Validation:

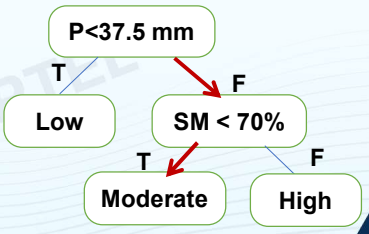
Tree 1:

The OOB observation for Tree 1 is 4, i.e., $P = 55mm$ and $SM = 65\%$

Actual class is Moderate.

Pass observation 4 through Tree 1:

- The observation reaches the leaf Moderate.
- Therefore, Tree 1 OOB prediction is correct



Similarly, the OOB predictions of Tree 2 and Tree 3 are computed and tabulated (shown in the next slide).

Solution

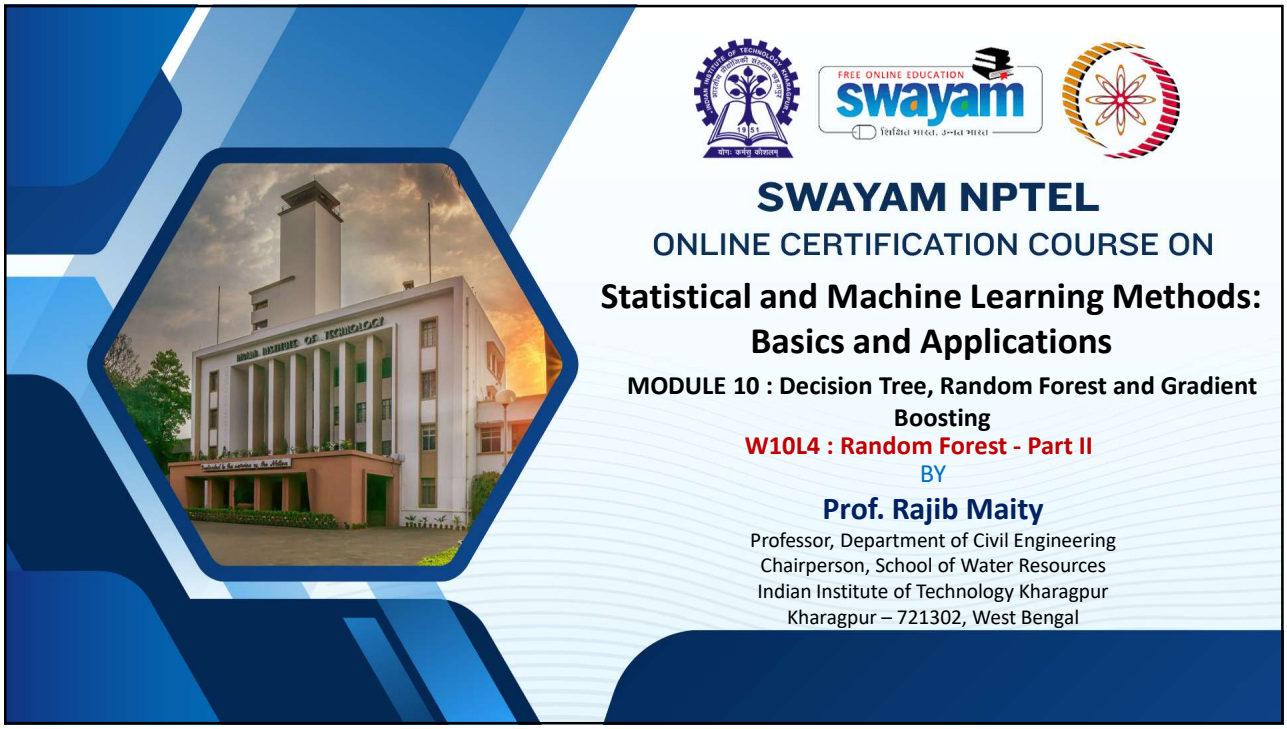
OOB Validation:

Tree	OOB observation	Actual class	Predicted class	Result
Tree 1	4	Moderate	Moderate	Correct
Tree 2	2	Low	Low	Correct
Tree 3	1	Low	Low	Correct

- No. of incorrect predictions = 0
- Total OOB predictions = 3
- OOB misclassification rate = $\frac{0}{3} \times 100 = 0\%$

(The zero OOB error results from this very simple six-observation example)


FREE ONLINE EDUCATION
swayam
Thank You!
alt



SWAYAM NPTEL

ONLINE CERTIFICATION COURSE ON

Statistical and Machine Learning Methods: Basics and Applications

MODULE 10 : Decision Tree, Random Forest and Gradient Boosting

W10L4 : Random Forest - Part II

BY

Prof. Rajib Maity

Professor, Department of Civil Engineering
Chairperson, School of Water Resources
Indian Institute of Technology Kharagpur
Kharagpur – 721302, West Bengal

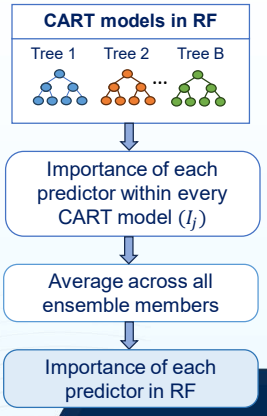
CONCEPTS COVERED

Random Forest: Some more details

NPTEL

Importance of Predictors

- A CART model can identify which predictors contribute most strongly towards reducing prediction error during tree construction.
- For regression, predictor importance is evaluated from the reduction in mean squared error (MSE) produced by each selected split.
- For classification, the corresponding reduction is calculated using Gini impurity or entropy impurity instead of MSE.
- The predictor importance approach developed for CART can be extended across the ensemble to estimate the predictor importance in Random Forest (RF).



Importance of Predictors in RF

- In the lth CART tree of the RF, the importance $I_j^{(l)}$ of predictor x_j is obtained by summing the error reductions from all splits using that predictor.
- For RF, the importance I_j for predictor x_j is then calculated as the mean of $I_j^{(l)}$ across all B ensemble members.

$$I_j = \frac{1}{B}\sum_{l=1}^{B} I_j^{(l)}$$

where, I_j is the average importance of predictor x_j across the complete ensemble and

$$I_j^{(l)} = \sum_{t=1}^{L-1} \Delta E_t \Im(b_t = x_j)$$

where, $\Delta E_t = E_t - E_{tL} - E_{tR}$ and $\Im(\cdot) = 1$ if the argument $(\cdot)$ is true and $\Im(\cdot) = 0$ if the same is false.

Alternative measure of Predictor Importance

- Surrogate splits may also be included in the CART models while estimating predictor importance within the Random Forest.
- An alternative importance measure can also be computed by calculating the Random Forest model error using the original OOB data.
  - For predictor x_j, its values in the OOB data are randomly permuted while the remaining data are unchanged.
  - The Random Forest model error is then recalculated using the OOB data containing the permuted predictor values.

Alternative measure of Predictor Importance

- The importance I_j, measure of j^{th}predictor, is defined as the difference between the permuted OOB error and the original OOB error.

$$I_j = E_{permuted\ OOB, x_j} - E_{original\ OOB}$$

- This permutation-based importance measure differs from the importance obtained by averaging across the ensemble members, although both measures usually give consistent results.

Relative Importance of Predictors

- The relative importance of predictor x_j is given by,

$$Relative\ importance\ of x_j = \frac{I_j}{\max(I_j)} \times 100\%$$

- The strongest predictor is assigned a relative importance of 100%.
- That is, the importance of the remaining predictors is measured relative to the strongest predictor.

Example:

A Random Forest regression model is developed to predict daily runoff using four predictors: 3-day rainfall, (P), Antecedent volumetric soil water content (SM), Air temperature (T) and Baseflow (BF). The training dataset and three CART regression trees fitted to predetermined training samples are provided in the following slides.

Using the importance of each predictor within every CART tree:

(a) Estimate the average importance of each predictor across the Random Forest model.

(b) Calculate the relative importance of all predictors.

Example:

Observation	P (mm)	SM (%)	T (°C)	BF (m^3/s)	Runoff, y (mm)
1	8	16	30	2.0	4
2	12	18	29	2.5	5
3	16	20	28	3.0	6
4	20	23	27	3.0	8
5	24	22	29	3.5	10
6	28	26	28	4.0	12
7	32	27	27	4.0	14
8	36	30	26	4.5	17
9	40	29	25	5.0	20
10	44	33	24	5.0	23
11	48	35	25	5.5	26
12	52	37	24	6.0	30
13	56	36	23	6.0	34
14	60	40	22	6.5	38
15	64	42	21	7.0	42

Example:

The given RF model contains three CART regression trees fitted to three predetermined training samples.

Tree	Training sample	OOB
Tree1	1,2,3,4,5,6,7,8,9,10	11,12,13,14,15
Tree2	1,2,3,6,7,8,9,12,13,14	4,5,10,11,15
Tree3	3,4,5,6,9,10,11,12,14,15	1,2,7,8,13

Tree1: Error reduction at each split

Root node:

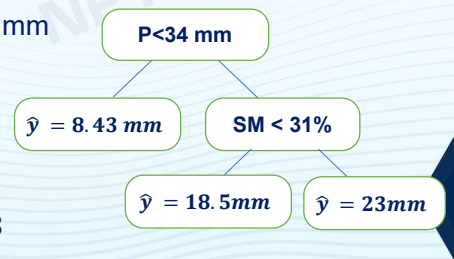
$y = \{4,5,6,8,10,12,14,17,20,23\}$, $\bar{y} = 11.9$ mm

$$E_t = \frac{\sum(y_i - \bar{y})^2}{10} = \frac{382.90}{10} = 38.29$$

Split 1: $P < 34mm$

$E_{tl} = 11.96$, $E_{tR} = 6.00$

$$\Delta E_P = 38.29 - 11.96 - 6.00 = 20.33$$



Example:

Split 2: $SM < 31\%$

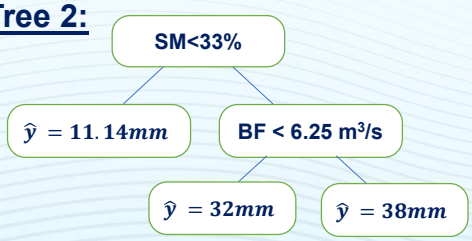
$E_t = 6.00;\ E_{tl} = 2.25,\ E_{tR} = 0$

$\Delta E_P = 6 - 2.25 - 0 = 3.75$

Tree-1	
Predictor	Importance
P	$I_P^{(1)}$ = 20.33
SM	$I_{SM}^{(1)}$ = 3.75
T	$I_T^{(1)}$ = 0
BF	$I_{BF}^{(1)}$ = 0

Similarly, the importance of each predictor within tree 2 and tree 3 are calculated and provided below.

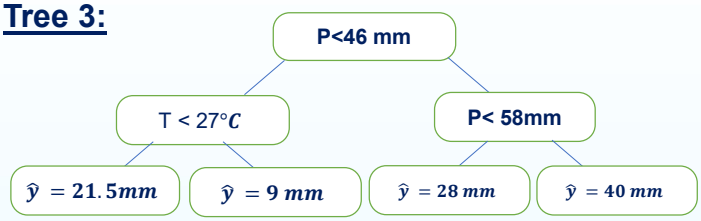
Tree 2:



Predictor	Importance
P	$I_P^{(2)}$ = 0
SM	$I_{SM}^{(2)}$ = 92.10
T	$I_T^{(2)}$ = 0
BF	$I_{BF}^{(2)}$ = 6.67

Example:

Tree 3:



Predictor	Importance
P	$I_P^{(3)}$ = 96.64
SM	$I_{SM}^{(3)}$ = 0
T	$I_T^{(3)}$ = 31.56
BF	$I_{BF}^{(3)}$ = 0

Solution:

(a) Importance of jth predictor across Random Forest:

$$I_j = \frac{1}{B}\sum_{l=1}^{B} I_j^{(l)}$$

Given: No. of trees, $B = 3$

Solution:

Therefore,

$$I_j = \frac{1}{3}\sum_{l=1}^{3} I_j^{(l)}$$

Predictor	RF Importance
Rainfall	$I_P = \frac{20.33 + 0 + 96.64}{3} = 38.99$
Soil Moisture	$I_{SM} = \frac{3.75 + 92.10 + 0}{3} = 31.95$
Temperature	$I_T = \frac{0 + 0 + 31.56}{3} = 10.52$
Baseflow	$I_{BF} = \frac{0 + 6.67 + 0}{3} = 2.22$

Solution:

(b) Relative importance of predictors:

- The relative importance of predictor x_j is given by,

$$Relative\ importance\ of x_j = \frac{I_j}{\max(I_j)} \times 100\%$$

- The strongest predictor (Rainfall) is assigned a relative importance of 100%.

Predictor	RF Importance	Relative Importance
Rainfall	38.99	100%
Soil Moisture	31.95	$= \frac{31.95}{38.99} \times 100 = 81.9\%$
Baseflow	10.52	$= \frac{10.52}{38.99} \times 100 = 27.0\%$
Temperature	2.22	$= \frac{2.22}{38.99} \times 100 = 5.7\%$

Quantile Regression Forest

- Random Forest regression models can be modified to perform quantile regression as well.
- In Random Forest, each leaf node on a tree contains at least n_{min} training observations.
- For regression, the usual RF prediction uses only the mean response value of the observations in a leaf node.
- Therefore, the other statistics of the response value available in that leaf node are not fully used in standard RF regression.
- Quantile estimates can be obtained from the conditional cumulative distribution function of the response variable.

$$F(y|x) = P(Y \leq y|x)$$

NPTEL

Quantile Regression Forest

- Here, $F(y|x)$ gives the probability that the response Y is less than or equal to y, given the predictor value x.
- For a given predictor value x, the corresponding leaf node is reached in each tree, and the response values in that leaf provide information about $F(y|x)$.
- In quantile regression forest (QRF), the response values in the reached leaf nodes are combined across trees to estimate the conditional quantiles.
- The procedure is as follows:
  - First train a standard RF regression model.
  - For a given predictor value of x, each trained tree is followed until the corresponding leaf node is reached.
  - The response values present in that leaf node provide information about the conditional distribution $F(y|x)$.

NPTEL

Quantile Regression Forest

- This information is then combined across all trees in the Random Forest to estimate the conditional distribution more accurately.
- Once $F(y|x)$ is estimated, the required conditional quantile $q_\alpha(x)$ can be obtained.
- For example, a 95% prediction interval $I(x)$ can be provided for the RF model output using the 2.5th and 97.5th conditional quantiles.

$$I(x) = [q_{0.025}(x), q_{0.97}(x)]$$

Applications of Random Forest

- RF has been used for probabilistic quantitative precipitation forecasting using numerical weather prediction (NWP) model outputs.
- RF has also been applied to estimate atmospheric pollutant concentrations, including PM2.5 over large regions.
- In hydrology, RF has been used for flood hazard risk assessment and ranking of hydrological signatures that is, indices characterizing hydrologic behavior.
- RF has supported solar radiation forecasting, marine pollution prediction, ocean temperature retrieval, and agricultural yield prediction.


FREE ONLINE EDUCATION
swayam
Thank You!
alt



SWAYAM NPTEL

ONLINE CERTIFICATION COURSE ON

Statistical and Machine Learning Methods: Basics and Applications

MODULE 10 : Decision Tree, Random Forest and Gradient Boosting

W10L5 : Gradient Boosting

BY

Prof. Rajib Maity
Professor, Department of Civil Engineering
Chairperson, School of Water Resources
Indian Institute of Technology Kharagpur
Kharagpur – 721302, West Bengal

CONCEPTS COVERED

Gradient Boosting

NPTEL

Boosting

- Boosting is a family of ensemble methods that transforms weak learners, also called base learners, into a strong learner.
- Random Forests (RF) also convert weak learners to strong learners.
- However, unlike RF, boosting creates an ensemble of weak learners step-by-step, where the weak learners are constructed sequentially.
- In boosting, each new learner is developed to reduce the errors of the preceding learners.
- Boosting generally performs better than the RF approach for many problems, while RF still tends to remain competitive.
- Boosting has more hyperparameters to tune and may overfit when noise is present in the data.

Boosting

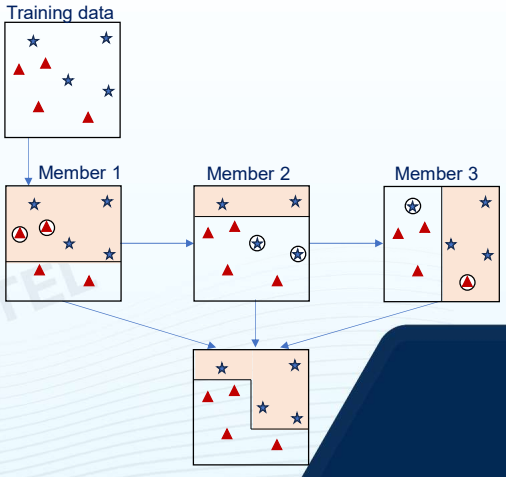
- Let the sequence of weak learners in a boosting model be $f_1(\mathrm{x}), f_2(\mathrm{x}),\ldots,f_M(\mathrm{x})$, where the ensemble contains M members.
- For an input $\mathbf{x}$, the jth member of the ensemble produces the output $f_j(\mathrm{x})$, where j ranges from 1 to M.
- Suppose the training data consists of predictor values (x_i) along with their corresponding target values (y_i), for $i = 1,2,\cdots,N$.
- After the learner $f_{j-1}(\mathrm{x})$ has been constructed, the next learner $f_j(\mathrm{x})$ is trained with greater emphasis on the observations x_i for which the difference between $f_{j-1}(\mathrm{x}_i)$ and the target y_i is relatively large.
- In this way, boosting attempts to improve the fit for observations that were difficult for the preceding learner.

Boosting

- Boosting combines the outputs of several weak learners to obtain the final ensemble prediction.
- For binary classification, the final class is determined from the majority vote of $f_1(x), f_2(x),\ldots,f_M(x)$.
- Conversely, for regression problems, the final prediction is calculated from the mean of $f_1(x), f_2(x),\ldots,f_M(x)$.
- The sequential development of a strong learner from several weak learners is illustrated using a binary classification example (in the following slides).

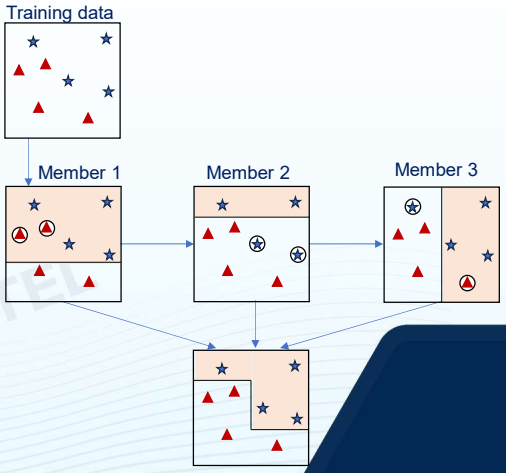
Boosting for Binary Classification

- The working principle of boosting in binary classification can be illustrated using a sequence of 3 members (simple weak learners).
- The top panel contains eight input observations belonging to two classes, ★ and ▲.
- Member 1 is then constructed using a simple decision tree, which divides the predictor space into two regions.
- The shaded region predicts one class, while the unshaded region predicts the other.



Boosting for Binary Classification

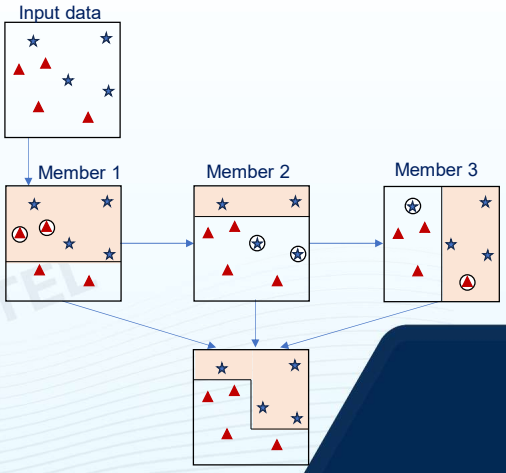
- Two observations fall within the wrong decision region and are therefore misclassified; these points are highlighted in the figure.
- While constructing Member 2, greater emphasis is placed on these two previously misclassified observations.
- As a result, they are classified correctly by Member 2, although two observations from the other class are now misclassified.



NPTEL

Boosting for Binary Classification

- Member 3 is then developed with increased attention to these newly misclassified observations.
- These points are correctly classified, but another two observations become misclassified.
- Finally, the predictions from the three members are combined using majority voting.
- The resulting ensemble produces a more complex decision region, shown in the bottom panel, in which all eight observations are classified correctly.



NPTEL

Gradient Boosting

- Gradient Boosting interprets boosting as an optimisation of an objective function.
- At each stage, a gradient-descent approach is used to improve the solution obtained from the preceding stage.
- Gradient boosting, also known as gradient tree boosting, has become a widely used boosting approach.
- Gradient boosting can be used in both regression and classification tasks.
- It improves the solution at stage j by applying gradient descent to the model obtained at stage $j-1$.

NPTEL

Gradient Boosting

- Let $\mathbf{x}_i$ represent the predictor values and y_i represent the corresponding target response, where $i = 1, \ldots, N$.
- The predicted output $f(\mathrm{x}_i)$ is compared against the observed target value using an objective or loss function (J).
- For instance, for regression problems, the loss function (J) can be the mean squared error:

$$J(f) = \sum_{i=1}^{N} J\big(y_i, f(\mathrm{x}_i)\big) = \sum_{i=1}^{N} \big(y_i - f(\mathrm{x}_i)\big)^2,$$

where, $f = (f(\mathrm{x}_1), \ldots, f(\mathrm{x}_N))$

NPTEL

Gradient Boosting

- The function $\boldsymbol{f}$ can be treated as a vector of adjustable parameters, that must be optimized to minimize the objective function, with the optimal f being

$$\hat{\boldsymbol{f}} = \arg\min_{\boldsymbol{f}} J(\boldsymbol{f})$$

- For classification problems, instead of mean squared error, other entropy-based loss functions, such as entropy impurity or Gini Impurity Index can be used.

Gradient Boosting

- At the stage j, let $\boldsymbol{g}_j$ be the gradient of $J(\boldsymbol{f})$ evaluated at $f = f_{j-1}$.
- The gradient for the ith observation is written as:

$$g_{ij} = \left[\frac{\partial J\left(y_i, f(\mathrm{x}_i)\right)}{\partial f(\mathrm{x}_i)}\right]_{f(\mathrm{x}_i)=f_{j-1}(\mathrm{x}_i)}, (i = 1, \ldots, N)$$

- The functional gradient descent updates the model from stage $j-1$ to stage j as

$$f_j = f_{j-1} - \gamma_j \boldsymbol{g}_j$$

  - Here, γ_j is the step length, determined from a line search,

$$\gamma_j = \arg\min_{\gamma} J(\boldsymbol{f}_{(j-1)} - \gamma \boldsymbol{g}_j)$$

Gradient Boosting

- The current solution is not sufficient because it optimizes f only at the N training observations.
- A functional form is therefore required to provide predictions for new values of $\mathbf{x}$.
- To achieve this generalization to new predictor values, a simple predictive model, referred to as a weak learner, such as a CART model, is fitted.
- This weak learner is used to approximate the negative gradient function.
- The weak learner $\phi(\mathrm{x}_i;\theta)$ with adjustable parameters θ, and the optimal parameters of the weak learner are obtained by:

$$\theta_j = \arg\min_{\theta} \sum_{i=1}^{N} \left(-g_{ij} - \phi(\mathrm{x}_i;\theta)\right)^2$$

Gradient Boosting Algorithm

The gradient boosting algorithm can be summarized as follows:

i. Initialization

Start by fitting a constant function $f_0(\mathrm{x}) = c$ to the target data y_i, that is,

$$f_0(\mathrm{x}) = \arg\min_{c} \sum_{i=1}^{N} J(y_i, c)$$

ii. For $j = 1, \ldots, M$, repeat the following steps.

a) Compute the gradient

$$g_{ij} = \left[\frac{\partial J\left(y_i, f(\mathrm{x}_i)\right)}{\partial f(\mathrm{x}_i)}\right]_{f(\mathrm{x}_i)=f_{j-1}(\mathrm{x}_i)}, (i = 1, \ldots, N)$$

Gradient Boosting Algorithm

b) Estimate the optimal parameters θ_j of the weak learner ϕ, so that it approximates the negative gradient, i.e., $-g_{ij}$:

$$\theta_j = \arg\min_{\theta} \sum_{i=1}^{N} \left(-g_{ij} - \phi(\mathrm{x}_i; \theta)\right)^2$$

c) Determine the optimal step length γ_j using a line search:

$$\gamma_j = \arg\min_{\gamma} \sum_{i=1}^{N} J\left(y_i, f_{j-1}(\mathrm{x}_i) + \gamma\phi\left(\mathrm{x}_i; \theta_j\right)\right);$$

d) Update the model, $f_j(\mathrm{x}) = f_{j-1}(\mathrm{x}) + \nu\gamma_j\phi\left(\mathrm{x}; \theta_j\right)$
with ν a learning rate hyperparameter.

iii. Finally, model output obtained after completing M boosting stages is

$$f(\mathrm{x}) = f_M(\mathrm{x})$$

CART Trees in Gradient Boosting

- A weak learner ϕ in Gradient Boosting is often represented by a CART decision tree having L terminal, or leaf, nodes.
- The predictor space is therefore divided into L separate regions.
- For CART-based Gradient Boosting, the step used to determine the common step length γ_j can be modified so that a separate step length is obtained for each terminal region of the tree.
- Instead of estimating a single value for the entire tree, a separate step length γ_{li} is determined for each terminal region l, where $l = 1, \ldots, L$.

CART Trees in Gradient Boosting

- For the lth region at boosting stage j,

$$\gamma_{lj} = \arg\min_{\gamma} \mathrm{x}_i \in R_{lj} \sum J(y_i, f_{j-1}(\mathrm{x}_i) + \gamma)$$

where R_{lj} represents the lth partitioned region during the jth boosting stage.

- Thus, rather than applying one common correction to the complete tree, each terminal region can have its own optimum correction.

Number of Leaf Nodes

- The number of terminal nodes L is a hyperparameter of the Gradient Boosting model.
- A tree with only $L = 2$ leaf nodes is generally considered too small, whereas trees with more than about $L = 10$ leaf nodes are usually unnecessary.
- Relatively small trees are generally preferred, with the number of terminal nodes typically selected within the range $4 \leq L \leq 8$.
- Since the final Gradient Boosting model $f_M(\mathrm{x})$ is obtained from an additive combination of several decision trees, the relative importance of predictor variables can also be evaluated in a manner similar to that used for Random Forests.

Learning Rate and Shrinkage

- The learning rate hyperparameter ν controls the magnitude of the step taken during the gradient-descent optimization.
- Although $\nu = 1$ is theoretically possible, smaller values satisfying $0 < \nu < 1$ generally provide better performance.
- Relatively small learning rates are generally recommended, for example $\nu < 0.1$.
- However, when a smaller learning rate is adopted, more boosting stages are generally required.
- A commonly recommended strategy is therefore to first select a small value of ν, and then determine a suitable number of boosting stages (M) using early stopping.

Example:

Using the following dataset, apply Gradient Boosting to predict runoff Q. Use the loss function as $J = \frac{1}{2}(y - f)^2$ and learning rate, $\nu = 0.5$. Perform two boosting stages.

Observation	P, (x_1) (mm)	SM, (x_2) (%)	Runoff, Q (y) (mm)
1	15	10	5
2	25	30	9
3	35	20	5
4	45	40	9
5	55	15	11
6	65	35	15
7	75	25	11
8	85	45	15

Solution:

Given: The loss function $J = \frac{1}{2}(y - f)^2$ and learning rate, $\nu = 0.5$

Step 1: Initialization

- Fit a constant function $f_0(\mathrm{x}) = c$ to the y_i data, that is,

$$f_0(\mathrm{x}) = \arg\min_c \sum_{i=1}^{N} J(y_i, c)$$

- For the squared-error loss, the optimum constant is simply the mean value of the target variable (Q).
- Thus,

$$f_0(\mathrm{x}) = c = \bar{y} = \frac{5 + 9 + 5 + 9 + 11 + 15 + 11 + 15}{8} = 10$$

Solution:

Step 2: (a) Compute the gradient

$$g_{i1} = \frac{\partial J}{\partial f} = \frac{\partial(\frac{1}{2}(y-f)^2)}{\partial f} = (f_0 - y_i) \Rightarrow -g_{i1} = (y_i - f_0)$$

Therefore, for the given squared-error loss, the negative gradient is equal to the current residual.

Boosting Stage -1:

Obs.	y_i	f_0	$-g_{i1} = y_i - f_0$
1	5	10	−5
2	9	10	−1
3	5	10	−5
4	9	10	−1
5	11	10	+1
6	15	10	+5
7	11	10	+1
8	15	10	+5

Solution

$$-g_{i1} = [-5, -1, -5, -1, +1, +5, +1, +5]$$

(b) Obtain the optimal weak-learner parameters θ_j for approximating $-g_{i1}$.

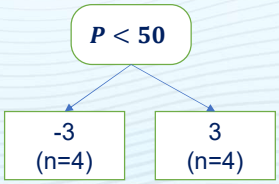
$$\theta_j = \arg\min_{\theta} \sum_{i=1}^{N} \left(-g_{ij} - \phi(\mathrm{x}_i; \theta)\right)^2$$

Obs.	P	SM	$-g_{i1}$
1	15	10	−5
2	25	30	−1
3	35	20	−5
4	45	40	−1
5	55	15	+1
6	65	35	+5
7	75	25	+1
8	85	45	+5

CART is fitted to these negative-gradient values. Among the candidate splits of P and SM, the minimum SSE is obtained for $P < 50$.

Left child: $\bar{y} = \frac{-5-1-5-1}{4} = -3$

Right child: $\bar{y} = \frac{5+1+5+1}{4} = 3$



Solution

Therefore, Tree 1 predicts

$$\phi_1(\mathrm{x}) = \begin{cases} -3, & P < 50 \\ +3, & P \geq 50. \end{cases}$$

(c) Determine the step length parameter γ_1 for first boosting stage from a line search:

$$\gamma_1 = \arg\min_{\gamma} \sum_{i=1}^{8} J[y_i, f_0(\mathrm{x}_i) + \gamma\phi(\mathrm{x}_i; \theta_1)]$$

(choose the value of γ that makes the total loss as small as possible)

We have, $f_0 = 10$; $\phi_1 = [-3, -3, -3, -3, +3, +3, +3, +3]$

Hence, the predictions are: $f_0 + \gamma_1\phi_1 = (10 + \gamma\phi_1)$

For the given squared-error loss,

$$\gamma_1 = \arg\min_{\gamma} \frac{1}{2} \sum_{i=1}^{8} [y_i - (f_0(\mathrm{x}_i) + \gamma\phi(\mathrm{x}_i))]^2$$

Solution

For the given observed values, $y = [5,9,5,9,11,15,11,15]$,

Thus,

$$\gamma_1 = \arg\min_{\gamma} \frac{1}{2}\sum_{i=1}^{8}[(5-(10-3\gamma))^2 + (9-(10-3\gamma))^2 + \cdots + (15-(10+3\gamma))^2]$$

$$\Rightarrow \gamma_1 = \arg\min_{\gamma}[52 - 72\gamma + 36\gamma^2]$$

The value of γ is obtained where the function is minimum.

$$\frac{dJ(\gamma)}{d\gamma} = 0 \Rightarrow -72 + 72\gamma = 0$$

Thus, $\gamma_1 = 1$

NPTEL

Solution

(d) The update is, $f_1(\mathrm{x}) = f_0(\mathrm{x}) + \nu\gamma_1\phi_1(\mathrm{x})$

$$\Rightarrow f_1 = 10 + 0.5(1)\phi_1$$

Obs.	y_i	f_0	ϕ_1	f_1	$-g_{i2} = y_i - f_1$
1	5	10	-3	8.5	−3.5
2	9	10	-3	8.5	+0.5
3	5	10	-3	8.5	−3.5
4	9	10	-3	8.5	+0.5
5	11	10	3	11.5	-0.5
6	15	10	3	11.5	+3.5
7	11	10	3	11.5	-0.5
8	15	10	3	11.5	+3.5

Solution

Boosting Stage - 2:

(a) Compute the gradient

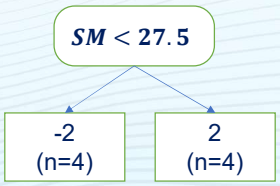
$$-g_{i2} = [-3.5, +0.5, -3.5, +0.5, -0.5, +3.5, -0.5, +3.5]$$

(b) Obtain the optimal weak-learner parameters θ_j for approximating $-g_{i2}$.

A new CART tree is fitted to these updated negative-gradients. Among the candidate splits of P and SM, the minimum SSE is obtained for $SM < 27.5$.

Left child: $\bar{y} = \frac{-3.5-3.5-0.5-0.5}{4} = -2$

Right child: $\bar{y} = \frac{0.5+0.5+3.5+3.5}{4} = 2$

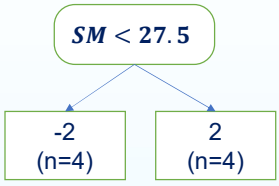


Solution

Boosting Stage - 2:

Therefore, Tree 2 predicts

$$\phi_2(x) = \begin{cases} -2, & SM < 27.5 \\ +2, & SM \geq 27.5 \end{cases}$$



Note: Two-leaf trees are used here only for illustration. In Gradient Boosting, the number of leaf nodes L is a hyperparameter, and larger weak learners may be used.

(c) Step length is calculated for this boosting stage similar to stage-1 and obtained as, $\gamma_2 = 1$.

Solution

(d) The update is, $f_2(\mathrm{x}) = f_1(\mathrm{x}) + \nu\gamma_2\phi_2\,(\mathrm{x}) = f_1(\mathrm{x}) + 0.5(1)\phi_2$

$$\Rightarrow f_2 = f_0(\mathrm{x}) + \nu\gamma_1\phi_1\,(\mathrm{x}) + \nu\gamma_2\phi_2\,(\mathrm{x})$$

Obs.	y_i	f_0	ϕ_1	f_1	ϕ_2	f_2
1	5	10	-3	8.5	-2	7.5
2	9	10	-3	8.5	2	9.5
3	5	10	-3	8.5	-2	7.5
4	9	10	-3	8.5	2	9.5
5	11	10	3	11.5	-2	10.5
6	15	10	3	11.5	2	12.5
7	11	10	3	11.5	-2	10.5
8	15	10	3	11.5	2	12.5

Solution

Reduction in error:

Initial residuals: $[-5, -1, -5, -1, +1, +5, +1, +5]$.

$$SSE = 25 + 1 + 25 + 1 + 1 + 25 + 1 + 25 = 104$$

$$J = 0.5 \times SSE = 52$$

Similarly, SSE and J are calculated for Tree-1 and Tree-2 and tabulated below.

	Initial	Tree 1	Tree 2
Residuals	$[-5, -1, -5, -1, +1, +5, +1, +5]$	$[-3.5, +0.5, -3.5, +0.5, -0.5, +3.5, -0.5, +3.5]$	$[-2.5, -0.5, -2.5, -0.5, +0.5, +2.5, +0.5, +2.5]$
SSE	104	50	26
J	52	25	13

Solution

- Tree-1 captures the dominant rainfall-related pattern in the initial errors.
- After updating the model, Tree-2 is fitted to the remaining errors and identifies soil moisture as the better split.
- Thus, each boosting stage learns a correction to the model obtained from the preceding stage.

Subsampling in Gradient Boosting

- Random Forest uses bagging, through which different bootstrap samples are used to construct the individual ensemble members. Gradient Boosting can similarly introduce randomness through subsampling.
- At each boosting stage, only a randomly selected fraction η of the training observations is used to construct the next tree; the sampling is performed without replacement, a typical suggested value is $\eta = 0.5$.
- Subsampling is generally more effective when it is used together with shrinkage, that is, with a relatively small learning rate ν.
- Similar to bagging, subsampling leaves some observations unused while constructing the next weak learner.
- These unused observations are used to estimate an out-of-bag validation error, which can help in selecting appropriate hyperparameter values.

Hyperparameters of Gradient Boosting

- As compared with the Random Forest approach, Gradient Boosting may achieve greater predictive accuracy, but it also contains more hyperparameters that require tuning.
- The main hyperparameters are:
  - L: Number of terminal nodes in each tree
  - M: Number of boosting stages
  - ν: Learning rate
  - η: Fraction of training observations used at each stage
- Suitable preliminary values of L, ν, and η can be first identified, while M can then be tuned more carefully as the main hyperparameter.

NPTEL

Further advancement of boosting methods

- More efficient variants, such as XGBoost and LightGBM, have been developed.
- These are particularly suitable for handling large datasets efficiently.
- CatBoost is another variant of boosting.
- This is especially useful when the dataset contains categorical predictor variables.

NPTEL


FREE ONLINE EDUCATION
swayam
Thank You!
alt